

GPS & Location-Based Mechanics

Kursüberblick

1. Intro
2. Mobile UX & Input
3. Performance & Mobile Constraints
-  4. GPS & Location-Based Mechanics
5. Map Integration
6. AR Foundation
7. Game Loop & Systems
8. Polishing + Smartphone Features
9. Prüfung

Heute - Theorie

- Location Service in Unity
- Android Permissions
- Koordinatensystem (Lat/Long)
- Distanzberechnung
- Mock Locations fürs Testing

Heute - Praxis

Locations tracken und mocken

- Tracken der Spielerposition
- Distanzberechnung

Gamifizieren des Location-Tracking

- Wenn nah genug an Creature, dann fangen!

Location Service in Unity

Location Service

Wir wollen einen Taschenmonster-Los Klon bauen!

Dafür brauchen wir:

- Creatures in der Welt spawnen
- Die Spielerposition
- Die Distanz vom Spieler zur Creature

Android hat dafür Tools

Coarse Location vs Fine Location

Coarse

- via Mobilfunk / WLAN Daten
- ungenau -> ~100m - 4km
- sparsamer im Batterieverbrauch
- für: Wetter, lokale News

Fine

- via GPS
- genau 2m - 20m
- höherer Batterieverbrauch
- für: Navigation, **Location-based Games**

Location Service in Unity

Und wie kommt man da jetzt ran?

Unity bietet einen Location Service an:

<https://docs.unity3d.com/ScriptReference/LocationService.html>

Dieser wird ganz einfach über `Input.location` genutzt.

1. Zum Starten: `Input.location.Start();`
2. Statusabruf: `Input.location.status`
3. Datenabruf `Input.location.lastData`

Location Service in Unity

Script erstellen:

1. Start Location Service Coroutine
2. TMPro Textfeld machen
3. Player Position schreiben

```
void UpdatePlayerPosition()
{
    float latitude = Input.location.lastData.latitude;
    float longitude = Input.location.lastData.longitude;
    float altitude = Input.location.lastData.altitude;

    _coordinateTextField.text = "Lat: " + latitude + "\n Long: " + longitude + "\n Alt: " + altitude;
}
```

```
IEnumerator StartLocationService()
{
    Input.location.Start(1f, 1f);

    int maxWait = 20;
    while (Input.location.status == LocationServiceStatus.Initializing && maxWait > 0)
    {
        yield return new WaitForSeconds(1);
        maxWait--;
    }

    if (Input.location.status == LocationServiceStatus.Running)
    {
        Debug.Log("Location ready");
        InvokeRepeating("UpdatePlayerPosition", 1f, 1f);
    }

    if (Input.location.status == LocationServiceStatus.Failed)
    {
        Debug.LogError("Shit!");
    }
}
```

Nix passiert!

Es passiert nichts! Wenn wir den Status debuggen: Status ist "Failed"!

```
Input.location.Start(1f, 1f);  
Debug.Log(Input.location.status);
```

Weil die GPS Berechtigungen fehlt!

```
if (!Input.location.isEnabledByUser)  
{  
    Debug.Log("User has GPS turned off in Android Settings");  
    yield break;  
}
```

Android Permissions

Bei *kritischen* Berechtigungen muss immer erst *zur Laufzeit* angefragt werden.
Keine pauschale Berechtigung bei Installation mehr.

Gilt für:

- Location über WLAN/Mobilfunk (*COARSE_LOCATION*)
- Location über GPS (*FINE_LOCATION*)
- Camera (*CAMERA*)
- Mikrofon (*RECORD_AUDIO*)
- Kontakte, Kalender, Sensoren
- Zugriff auf Medien, Zugriff auf externen Speicher

Android Permissions anfragen

Zunächst neuen Namespace einbinden: `using UnityEngine.Android;`

Dann die Anfrage hinzufügen.

- Nur wenn wir die Berechtigung noch nicht haben

```
// Check if permission is already granted
if (!Permission.HasUserAuthorizedPermission(Permission.FineLocation))
{
    Permission.RequestUserPermission(Permission.FineLocation);
    // Wait for the user to click "Allow"
    yield return new WaitForSeconds(2);
}
```

Geht immer noch nicht!

Unser PC / Laptop hat keinen GPS Empfänger.

Also müssen wir einen Build machen.

Testen auf Android!

Okay, klappt!

Aber was genau lassen wir uns da eigentlich ausgeben?

Ich hoffe ihr mögt Erdkunde!

Kurze Pause?

Exkurs - Erdkunde I

Latitude - Breitengrade

von -90 bis +90

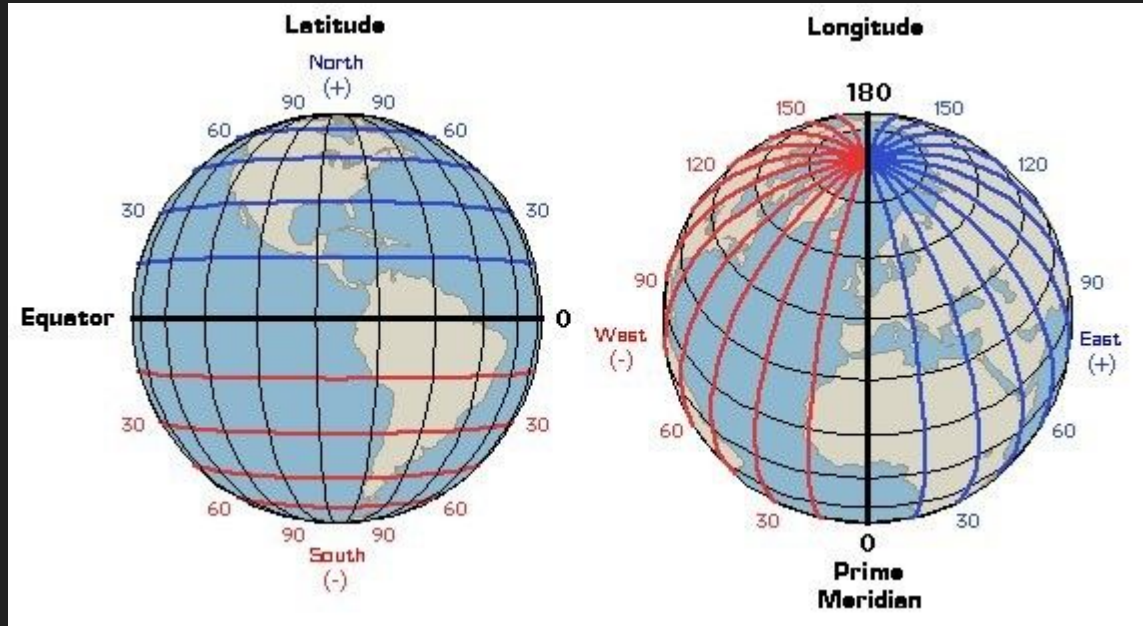
Äquator als 0

Longitude - Längengrade

von -180 bis +180

Nullmeridian als 0

Greenwich, London



Exkurs - Erdkunde II

Benutzung:

Lat: Negativ -> Süd, Positiv -> Nord

Long: Negativ -> West, Positiv -> Ost

Position SRH:

49.414, 8.650

49 = N, 8 = E

APIs (z.B. Google Maps) wollen meistens (Lat, Long)

Exkurs - Erdkunde III - nice to know

Zwei Systeme:

- 49.414 - 8.650 (Dezimalgrad, DD)
- 49°24'50.8"N - 8°39'03.5"E (Klassische Format, DMS)

Umrechnung:

49° bleibt, für die Nachkommastellen:

0.414 x 60 (Minuten) -> 24.84 -> 0.84

0.84 x 60 (Sekunden) -> 50.4

= 49.414 -> 49°24'50''

Optimierung

Okay, klappt!

Aber! Constant GPS Polling killt die Batterie! -> Optimieren

```
Input.location.Start(1f, 1f);
```

 Was heißt das eigentlich?

```
public void Start(float desiredAccuracyInMeters, float updateDistanceInMeters);
```

`desiredAccuracyInMeters` => Wie genau wollen wir sein?

- bei 10m -> GPS Chip only
- bei 500m -> fast nur über WLAN/Mobilfunk
- dazwischen -> Mischung

`updateDistanceInMeters` => Wie oft brauchen wir Updates?

(wieviel Meter muss sich das Handy bewegen, bevor Unity das nächste mal Input.Location updatet)

- kleiner = mehr Updates = mehr Batterieverbrauch

GPS - Optimierung

1. Option: höhere Werte einstellen: `Input.location.Start(10f, 10f);`
 - a. für uns sollten 10m/10m reichen
2. Option: Werte dynamisch einstellen
 - a. Verschiedene Werte je nachdem, was im im Spiel tun:
 - i. Idle: 100m/50m
 - ii. Gehen: 10m/10m
 - iii. Creature Encounter: 5m/2m

```
enum TrackingMode
{
    1 reference
    Idle,
    1 reference
    Walking,
    0 references
    Encounter
}
```

```
void SetTrackingMode(TrackingMode mode)
{
    Input.location.Stop();

    switch (mode)
    {
        case TrackingMode.Idle:
            Input.location.Start(100f, 50f);
            break;
        case TrackingMode.Walking:
            Input.location.Start(10f, 10f);
            break;
        case TrackingMode.Encounter:
            Input.location.Start(5f, 2f);
            break;
    }
}
```

Distanzberechnung

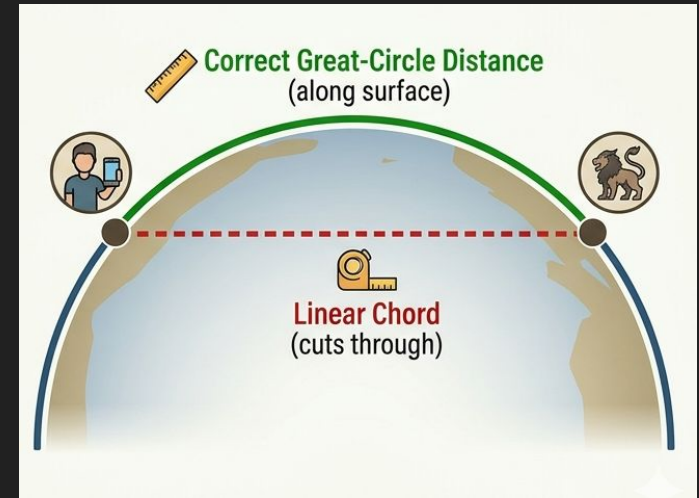
Distanzberechnung - Ab durch die Mitte

Wir brauchen die Distanz für:

- Entfernung zu Creatures anzeigen
- Spawnradius überprüfen
- Interaktion ab bestimmter Nähe erlauben

Problem: Wir sitzen auf einer Kugel (wirklich!)

- $d = y - x$ gibt verfälschte Ergebnisse
- wir schneiden durch die Kugel



Distanzberechnung - Haversine

Gibt eine simple Formel, die Haversine:

$$d = 2r \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta\varphi}{2} \right) + \cos(\varphi_1) \cos(\varphi_2) \sin^2 \left(\frac{\Delta\lambda}{2} \right)} \right)$$

So, dann einmal implementieren!

Distanzberechnung - Doch nicht die Haversine?

Halt - Stopp!

Oberflächendistanz	Abweichung durch Schneiden der Kugel
1.000 km	~ 100m - 1km
100 km	~ 10m - 100m
10 km	~ 1mm - 1cm
1 km	< 0 mm

Können wir ignorieren!

Kreise machen Probleme

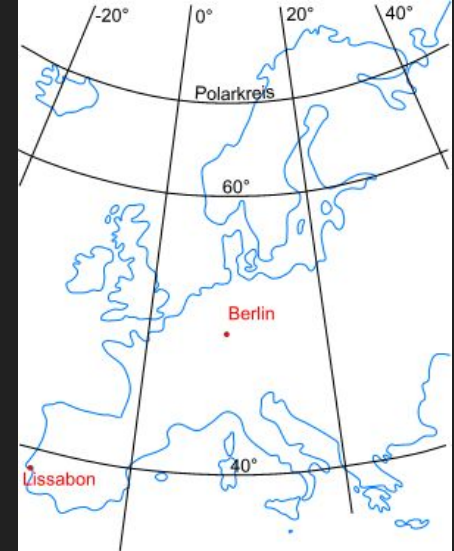
Es gibt aber ein weiteres Problem: Verzerrung durch die Kugelform

Distanz zwischen einzelnen Graden:

- Abstand zwischen zwei Breiten: konstant 111,3km
- Abstand zwischen zwei Längen:
 - am Äquator: 111,32km
 - an den Polen: 0m
 - und in Polen? $\sim 52^\circ$ N

Formel : $111.32 * \cos(\text{lat})$

$111.32 * \cos(52^\circ) \Rightarrow 68,54 \text{ km}$



<https://www.kompf.de/gps/images/europe.png>

Distanzberechnung - stark vereinfacht

Distanz zwischen den Koordinaten:

```
float differenceDegree = Vector2.Distance(positionA, positionB);
```

Dann noch auf Meter umrechnen (mit Konstante 111,32km):

```
float distanceMeters = differenceDegree * _metersPerDegreeLat; // _metersPerDegreeLat = 111320f;
```

Für verbesserte Genauigkeit:

1. X und Y trennen, unterschiedliche

```
x = dLon * metersPerDegreeLong; //Formel: 111320f * cos(Lat) // = 68535f (m) für Polen
```

```
y = dLat * _metersPerDegreeLat; //Konstante 111320f
```

2. Anschließend Euklidische Distanz berechnen

```
Mathf.Sqrt(x * x + y * y); (Dies ist die gleiche Formel wie Vector2.Distance(x,y))
```

Aufgabe - 30 min

1. GPS Modi

- 3 Modi implementieren (Idle: 100/50; Walking: 10/10; Encounter: 5/2)
- Button implementieren, der die GPS Modi umschaltet.
- Anzeige der aktuellen Position

2. Distanzberechnung

- Zielkoordinate: 49.41399, 8.65110, SRH Eingangstür
- kontinuierliche Distanzberechnung und Anzeige im Textfeld
 - simple Distanz: `Vector2.Distance(positionA, positionB);`
 - (nur wer fertig ist!) Komplexere Distanz mit Umrechnung von Metern pro Längengrad*

```
void SetTrackingMode(TrackingMode mode)
{
    Input.location.Stop();

    switch (mode)
    {
        case TrackingMode.Idle:
            Input.location.Start(100f, 50f);
            break;
        case TrackingMode.Walking:
            Input.location.Start(10f, 10f);
            break;
        case TrackingMode.Encounter:
            Input.location.Start(5f, 2f);
            break;
    }
}
```

Tipps

Für verbesserte Genauigkeit:

1. X und Y trennen, unterschiedliche

```
x = dLon * metersPerDegreeLong; //Formel: 111320f * cos(Lat) // = 68535f (m) für Polen  
y = dLat * _metersPerDegreeLat; //Konstante 111320f
```

2. Anschließend Euklidische Distanz berechnen

```
Mathf.Sqrt(x * x + y * y); (Dies ist die gleiche Formel wie Vector2.Distance(x,y))
```

Kurze Pause?

Mock Locations

Mock Locations - Plattform Unterscheidung

Wir wollen im Editor testen können. Aber wie?

- Wenn **Android** -> **echte Daten nutzen**
- Wenn **Editor** -> **Mock Location Service nutzen**

Option a) Bool

```
if ( _useMockLocation)
    StartMockLocationService();
```

Option b) Compilation Directives

```
#if UNITY_EDITOR
    StartMockLocationService();
#else
    StartCoroutine("StartLocationService");
#endif
```


Mock Locations Service

Mock Location Service:

1. Starten: `InvokeRepeating("UpdatePlayerPositionMock", 1f, 1f);`
2. Updaten: statt `Input.location.lastData` einfach `_playerCoords.x = _mockedPlayerPosition.x;` abfragen

Für dynamisches Testen Zusatzfunktionen implementieren:

3. `PlayerPosition` während runtime überschreiben -> Teleport
4. Funktion für "laufen"
 - a. z.B. kontinuierlich nach Osten gehen. -> `_mockedPlayerPosition.y+= (2f/111320f); //für 2°s`

Aufgabe

Erweitern der Szene um Mock-Locations

1. Testen mit Mock Testing
 - a. 3 Buttons mit festen Mock Locations
 - b. ein Button mit -> "Walk East" -> 2m/s (°s = 2 / 111320)
 - c. ein Button mit -> "Walk North" -> 2m/s

Kurze Pause?

Gamify

Gamify the locations!

Umbauen unserer Debug Szene zu einem ersten “Game”:

- Creature an bestimmten Locations spawnen -> via Pos Buttons
- Spielerposition tracken (auf Performance achten!)
- Distanz zur “nächsten” Creature berechnen und anzeigen
- Wenn Distanz kleiner 2m dann Creature gefangen -> Load Creature Scene
- Ohne Karte, Simple UI.

1. Erst im Editor mocken
2. dann in Real-Life testen

Spawn and Track - Tipps

-